Site Intelligence Agent

ENMGT 5400 - Applications of Artificial Intelligence for Engineering Managers

Name: Nishchay Vishwanath

NetId: nv268

Cornell University, Master of Engineering in Engineering Management,

May 2026

1. Executive Summary

Some of the things that field technicians do when working in HVAC maintenance and drone-based site inspections is to routinely consult dozens of documents, equipment manuals, US regulatory OSHA standards, inspection records, and maintenance histories of those particular safety-critical devices. What they're looking for is an answer to safety-critical questions itself. The main pain point here is that the cost of a wrong answer is not inconvenience. It is possibly an equipment failure, regulatory violation, or health injury.

What this project does is deliver the Site Intelligence Agent. It's a retrieval-augmented generation pipeline that provides natural language answers in a grounded trustworthy manner. The answers here are grounded or also grounded in a curated document repository or corpus with an explicit graceful degradation layer that determines whether the system has enough evidence to answer or not and routes accordingly. Here are three routing parts that govern every response:

1. High-confidence: The system answers with source citations and gives the steps or gives the LLM output.
2. Partial Confidence: Here the system answers but flags a conflict or an uncertainty, escalating it to a human for review.
3. Low confidence. Here the system does not call the large language model at all. However what it does is return a deterministic escalation message explaining what was searched and recommending that the technician contact their office or supervisor.

Important note: This anti-hallucination design is the core architectural differentiator when compared to other rack systems. Other rack systems will pass every query to a large language model, regardless of retrieval quality. What it also does is that it trusts the model to stay grounded; however, that fails more often than not. This system treats low-confidence retrieval as a possible data signal and acts on it deterministically before the language model can hallucinate/confabulate.

2. Problem Framing and Business Case

2.1 The Field Technician's Information Problem

Let's create an instance slash a scene. A junior HVAC technician is on a building rooftop on a sunny day and mid repair he asks: “Hey what are the lockout/tagout steps for this carrier unit?” The answer exists essentially but it might be in a 400 pdf. sorry 400 page pdf document which might be in a binder in the office or in the car he is missing OSHA regulations and the job history of the last technician's visit. And then it takes around 8 to 12 minutes or maybe even 30 minutes to locate all of this manually in a safety critical situation. That delay is a painful pain point.

This exact problem exists in drone site inspections as well. A field operator reviewing a Zone C structural panel wants to know, “hey, is the corrosion I'm seeing above baseline?” - And this answer requires cross-referencing of current inspection records against historical baseline documents, which are two separate files with no automated comparison

Existing tools, however, fail and speak two specific ways:

1. Generic search engines return document links, not answers.
2. Generic LLMs return hallucinated answers. Hence, unguarded language model interfaces answer every question confidently, even if there is no context or corpus basis. For a question about a specific equipment model, the corpus is not there. The model will often give plausible-sounding, fabricated specifications, which might be detrimental in a field-critical environment.

The Site Intelligent Agent addresses both of these failure modes:

1. It retrieves from a curated domain-specific corpus
2. It refuses to answer when retrieval quality is insufficient

2.2 The Field Technician's Information Problem

Today hosted drag solutions do exist such as Azure AI search vertex AI search and others but the decision to build a custom pipeline was driven by 4 requirements that enterprise or commercial products do not meet out-of-the-box.

Requirement	Commercial RAG	Site Intelligence Agent
Explicit LOW-confidence escalation	Not available	Core feature — programmatic, no LLM call
Cross-collection conflict detection	Not available	detect_conflicts() heuristic

Requirement	Commercial RAG	Site Intelligence Agent
Domain-aware confidence thresholds	Not configurable	Per-domain tuning via env vars
Session memory + spatial filtering	Limited	Built in Phase 2

And the principal cost of building custom every time is approximately approximately 6 to 8 weeks of Phase 1 pipeline and eval work auto fully loaded engineering cost of may be 100 to $150.00 an hour the build investment is pretty high and against an enterprise field operations team of 50 technicians saving about 8 minutes per query at 10 queries per day this break even horizon is approximately 4 to 5 months.

The build versus by calculus shifts decisively towards build especially for use cases where the failure mode for wrong answer is measurably costly for example liability safety incidents regulatory fine.

2.3 ROI Framework

For the ROI framework, there are three quantifiable impact vectors:

1. Time to Answer Reduction. Manual document lookups take about 8 to 12 minutes. Rack queries takes less than 8 seconds. At 10 queries per technician per day across 50 technicians, there is approximately about 125 engineering hours recovered daily.
2. Escalation Cost Avoidance - each unnecessary escalation to a supervisor or vendor support costs time as well as money, an estimated $40 in loaded time. The system's low path escalations are targeted, evidence-based, and not reflexive, hence saving on this cost and time.
3. Liability risk reduction. A confident wrong answer on lockout or tagout procedures creates direct liability. The LOW path eliminates this class of response itself and hence saves a lot of cost.

3. System Architecture

3.1 Five-Step Pipeline

Every query flows through five steps in sequence. The first four are always executed. However, step four (LLM generation) is conditionally skipped on low-confidence retrievals.

```
1. private static int MAX = 5;
2. 1. 1.query
3. 2. 2.   → classify()         # classifier.py: query type + entity extraction
4. 3. 3.   → retrieve()         # retriever.py: embed query, search Chroma, top-k
```

```
5. 4. 4.   → score_confidence() # confidence.py: cosine similarity + conflict
6. 5. 5. detection
7. 6. 6.   → llm.generate()     # llm.py: called ONLY for HIGH or PARTIAL
8. 7. 7. confidence
9. 8. 8.   → route()            # degradation.py: format final response
```

The classifier step is present only in Phase 2 (drone domain). Phase 1 (HVAC) omits it, sending all queries directly to retrieval across all three HVAC collections. This intentional simplification kept Phase 1 verifiable against a clean baseline.

The classifier step here is present only in Phase II (Drone Domain). Phase I (HVAC) omits it, sending all queries directly to retrieval across all three HVAC collections. This intentionally simplifies Phase I verifiability against that clean play baseline.

3.2 Document Collections

There are six chroma vector database collections that are organized across two domains. These are never merged. Hence, the system is source-aware and can detect_conflicts, which means the system can identify which collection a result came from.

Domain	Collection	Documents	Chunks	Purpose
HVAC	osha	OSHA 1910.147, 1910.303	283	Lockout/tagout and electrical safety
HVAC	manuals	Carrier 48LC (2017, 2023), Lennox SL280, Trane XR15	1,372	Equipment specifications
HVAC	job_history	50 synthetic job records	181	Prior maintenance history
Drone	inspection_records	50 synthetic inspection records	202	Current anomaly findings per zone
Drone	historical_baselines	25 zone baseline documents	160	Normal operating thresholds
Drone	compliance_docs	OSHA 1926.452 (scaffold safety)	158	Regulatory compliance standards

3.3 Component Summary

Component	File	Role
Classifier	src/classifier.py	Rule-based gate + LLM fallback; routes to collections
Retriever	src/retriever.py	Embeds query, searches Chroma, applies spatial filter
Confidence Scorer	src/confidence.py	Thresholds top score; detects cross-source conflicts
Graceful Degradation	src/degradation.py	Formats HIGH/PARTIAL/LOW responses
LLM Wrapper	src/llm.py	Provider-agnostic (Anthropic / OpenAI / Gemini)
Session Memory	src/session_memory.py	Tracks zone/equipment/time across conversation turns
Ingest	src/ingest.py	Chunks documents, embeds, stores to Chroma

4. Evaluation Framework

The evaluation framework is part of this project, which is most likely to transfer to production context. This framework answers three distinct questions that are mainly necessary but not sufficient on their own. This makes the system product-ready, not just a demo.

4.1 Three-Set Methodology

Test Set 1: Ground Truth (50 queries)

The 50 ground truth queries test whether the system correctly answers known questions. The past criterion here is that the confidence level is LOW, and the system should return HIGH or PARTIAL for any questions answerable from the corpus of information. Failure in this stage means the system is under-indexing and escalating when it has the answers.

Test Set 2: Adversarial (20 queries)

The 20 adversarial queries test whether the system can correctly refuse answers to unknown questions. The pass criterion confidence level is set to LOW, and then every query targets equipment, regulations, or scenarios that do not exist in the corpus. Failure here is the most dangerous outcome: the system returns a PARTIAL or HIGH answer for a question it has no basis to answer.

Test Set 3: Contradictions (15 scenarios)

The 15 contradictions scenarios test whether the system can surface any sort of conflicts between sources rather than silently choosing one. The pass criterion here is the confidence level is equal to partial. Each scenario involves a question that should retrieve results from two different sources with conflicting or version-divergent information.

4.2 Evaluation Results

Test Set	Pass	Total	Pass Rate	Pass Criterion
Ground Truth	47	50	94.0%	confidence ≠ LOW
Adversarial	9	20	45.0%	confidence = LOW
Contradictions	13	15	86.7%	confidence = PARTIAL
Overall	69	85	81.2%	--

4.3 Ground Truth: 94% - Strong Baseline

The ground truth being 94% gave out three failures. All three share the same root cause. The Trane XR15 and Lennox SL280 manuals embed at lower similarity scores than OSHA documents. OSHA documents. Queries about equipment specifications occasionally fall into the low threshold even if the answer is present. This is a known limitation of dense vector retrieval. The fix here is some sort of hybrid retrieval: BIM 25 Keyword Matching + Dense Vector, deferred to Phase 3.

4.4 Adversarial: 45% - Accepted Limitation

For adversarial evals there were 11 failures. Every failure had the same pattern: the query targets equipment that does not exist in the corpus but a semantically near-miss equipment that does. An example of this is:

1. What is the refrigerant change charge for a Trane XR13?

This query scored 0.579, returned partial. The XR15 manual exists. XR13 and XR15 are similar enough that the embedding space does not discriminate them.

2. What are the OSHA requirements for confined space entry?

This query scored 0.762 and returned partial. The confined space context is adjacent to the lockout tagout corpus.

Implication for managers: Since there is a 45% adversarial ratio, it implies that PARTIAL answers will be returned in 55% cases. While in practice, it should be made clear to the operators that when it comes to answering with PARTIAL answers, further verification is necessary. It is better than not knowing about the queries at all.

4.5 Contradictions: 86.7% - Two Missed Conflicts

Two failure instances. Both deal with the issue of Carrier 48LC from 2017 vs. 2023 concerning certain maintenance interval specifications where the highest score for the extracted chunk does not exceed 0.50, resulting in LOW before the conflict detection process is even executed. The conflict detection algorithm is activated only if there is sufficient semantic information (score ≥ 0.50).

4.6 Threshold Tuning Methodology

The confidence thresholds are decisions about a product rather than engineering constants. They incorporate an inherent risk assessment: in a safety-critical domain, having an incorrect but confident result is riskier than being unnecessarily escalated to further processing.

These thresholds are configurable without modifying any source code:

```
1. 1. # Raise HIGH threshold to reduce hallucination risk:
2. 2. CONFIDENCE_HIGH_THRESHOLD=0.85 python demo/demo.py
3. 3. # Lower PARTIAL threshold to escalate more aggressively:
4. 4. CONFIDENCE_PARTIAL_THRESHOLD=0.60 python demo/demo.py
```

Current production parameter values: HIGH = 0.79, PARTIAL = 0.50. Found using eval tests for various combinations of threshold values and taking the combination that gave the best ground-truth success rate without having any adversarial attacks result in a HIGH failure.

4.7 Phase 2 Eval Strategy

Set	Target	Criterion
Drone ground truth (30 queries)	> 80% pass	confidence ≠ LOW
Drone adversarial (15 queries)	> 70% pass	confidence = LOW
Classifier eval (20 queries)	> 90% pass	correct query_type routing

This new classifier evaluation involves evaluation of query type routing (COMPLIANCE_LOOKUP, HISTORICAL_LOOKUP, ANOMALY_QUERY, OUT_OF_SCOPE) independent of retrieval performance, hence providing a way to debug any issues related to either classification or retrieval.

5. Graceful Degradation Layer

5.1 The Core Differentiator

The degradation layer, implemented in the src/degradation.py file, distinguishes our system from the regular RAG-based chatbots. The guiding principle behind its design is the fact that the system should never give a wrong answer. Engineering-wise, it means that the LOW confidence path doesn't make an API call to the language model.

Main guarantee: the LOW path doesn't make calls to the LLM. The escalation message is generated programmatically (deterministic output for the same input), which is key for operator trust.

5.2 Confidence Scoring

The routing is implemented through the score_confidence() method found in src/confidence.py file. This process takes into account three conditions: high score, conflicts, and number of strong results to route HIGH, PARTIAL, or LOW. In case there are no results from search, routing is performed LOW straight away. Conflicts override scores to route PARTIAL.

5.3 Conflict Detection

The detect_conflicts() algorithm applies two heuristics:

1. Heuristic 1: Cross-collection heuristic - detects discrepancies between OSHA and manual data. High-scoring records from distinct authoritative sources having a difference of no more than 0.15 in their scores generate a conflict.

2. Heuristic 2: Within-collection version conflict - detects conflicting versions of the data (for example, 2017 vs. 2023 Carrier 48LC manuals). High-scoring files within a single collection having a difference of no more than 0.15 in their scores generate a conflict.

5.4 Response Routing

These different routing paths yield different styles of output results as follows:

1. HIGH path: Answer with high confidence and citations embedded. Sources are cited directly (name of collection and source document). Confidence indicator precedes answer.
2. PARTIAL path: Answer with warning of contradiction. Two different sources are mentioned. Technician should check with the supervisor before proceeding.
3. LOW path: No LLM call out. Escalation deterministic message including statement of query, cause for escalation, nearest match if applicable (and score), and action recommended.

6. Phase 2: Drone Inspection Domain

6.1 Domain Extension Design

In phase 2, we show that the pipeline is domain independent. Introducing a new domain entails a new collection for Chroma, a classifier branch, and recalibration of the domain's confidence threshold. The scoring, conflict detection, and graceful degradation layers will remain unchanged.

The drone inspection domain introduces three elements to our architecture that are not part of Phase 1:

1. The Classifier Agent - forwards queries to the right collection before retrieval.
2. The spatial metadata filters - retrieval can be limited to certain zones or time windows.
3. Session memory - tracking zone/equipment/time instances through multiple turns.

6.2 Classifier Agent

Classifier in src/classifier.py acts as a gate before retrieval. Classifier implements a two-path architecture: the fast path processes around 70% of queries by applying rules without invoking any API; the second one is LLM-based structured output path for processing ambivalent queries.

Short-circuiting OUT_OF_SCOPE is vital for the application as it ensures that queries regarding weather forecasts, stock exchange rates, or any irrelevant subjects will not be processed by the retrieval and LLM components at all.

Spatial entities like Zone ID ("Zone-C"), Equipment Type ("rooftop-hvac"), and time references ("last month", "August 2025") are extracted from each query.

6.3 Spatial Filtering

Metadata level filtering is done when executing a query so that the results retrieved include only the relevant ones. Zone filtering applies only to inspection records since compliance and baseline documents are not zone-specific for each chunk. If a query with filters produces no results, the whole corpus will be searched as a fallback.

6.4 Session Memory

The SessionMemory object within src/session_memory.py allows anaphoric referencing between turns such that once the user inquires about "Zone C," any subsequent question of "What about last month?" refers to "Zone C" through session context and does not need further elaboration.

Session context will be constructed after the turn is classified (and not prior), so the entities from the current turn are what build the next turn's session context - not the current retrieval's.

7. AI Risk & Governance

7.1 Confidence Thresholds as Governance Decisions

Most importantly, the design principle which makes all the difference here is that confidence thresholds are a product choice made by subject-matter experts and not an engineering choice to optimize some hyperparameters. This is relevant when considering the implications for the engineering manager responsible for putting this system into production.

If an engineer was focused solely on optimizing for ground truth pass rate, she would tune the PARTIAL threshold down to make more queries fit into the HIGH or PARTIAL bucket, thereby optimizing coverage metrics. However, a domain expert aware of the domain context would know that, by doing this, they are increasing the likelihood of confidently getting it wrong regarding lockout/tagout procedures. They are different optimization goals.

The use of env vars makes the difference clear. Changes to thresholds need to be explicitly decided by humans, not coded in.

7.2 Hallucination Risk in Safety-Critical Contexts

Hallucination failures in RAG can occur in two ways:

1. Type 1 (failure of retrieval): No relevant documents are found within the corpus. The LLM produces an appropriate response based on pre-training, unrestricted by context retrieval. Solved via the LOW pathway, where in case of retrieval failure, the LLM is not activated.
2. Type 2 (contamination): The corpus contains relevant but not applicable texts. The LLM produces an appropriate but incorrect response, drawing from incorrect sources. Solved via the system prompt, which requires responses to be generated based on the context given and citing exact sources for the response.

The purpose of the adversarial testing dataset is to target Type 1 hallucination failures. System prompts and citations are used to address Type 2 failures. While neither problem can be said to be fully solved, as the adversarial pass rate stands at only 45%, both are well-controlled.

7.3 Human-in-the-Loop Design

PARTIAL is an example of a direct handover that exposes conflict details, identifies both origins of the conflict, and instructs the technician to seek approval from his/her supervisor before taking any action. This is not a contingency measure but rather a purposeful interaction that recognizes the epistemic uncertainty of the system and channels it to someone competent enough to address it.

The engineering manager implementing this system is responsible for specifying the escalation procedure: Whom should the technician contact? What is the Service Level Agreement (SLA)? How should the escalation be documented?

8. Deployment & Team Model

8.1 What Productionizing Requires

The current system runs locally on a laptop. Productionizing requires four infrastructure changes:

Component	Local (current)	Production
Vector database	Local Chroma directory	Chroma Cloud or Pinecone
LLM API	Single provider, env var	Multi-provider failover, rate limiting
Document ingestion	Manual CLI command	Automated pipeline on document upload
Serving	Streamlit development server	Containerized, behind load balancer

8.2 Team Model

At least three positions are necessary in order to develop this type of system:

1. ML Engineer (1 full time position): Responsible for maintaining the embedding model, confidence thresholds, conflict detection algorithms, evaluation frameworks. This person owns the pipeline.
2. Domain Expert/Knowledge Manager (0.5 full time positions): The individual responsible for controlling the document corpus - which documents will be included, how manuals will

be versioned, validating eval ground truth. Domain expert position is essential; quality of this system depends on the quality of its document corpus.

3. Product/Engineering Manager (0.5 full time positions): Decision-maker when it comes to thresholds, escalation rates, HIL workflow, tracking evolution of eval results.

Domain expert position is often underappreciated. Curating and managing document corpus, as well as updating eval ground truth is an ongoing operational task, not something you do once.

8.3 Build vs. Buy — Final Assessment

The build-versus-buy decision becomes more apparent after the first phase of implementation. While a hosted RAG system would have taken two weeks to accomplish the tasks of retrieval and generation, the custom-built option took six weeks but was able to deliver three functionalities that cannot be found in commercial systems: the LOW-path escalation feature, cross-source conflicts, and domain-aware thresholding.

For use cases in which an incorrect response carries measurable costs and the retrieval data set is under control, build. For use cases in which general search quality is sufficient and the retrieval data set is extensive and unorganized, buy.

9. What I Learned

9.1 The Eval Framework Revealed the Bug

The key event during Phase 1 was when the evaluation framework found a bug that could not have been discovered via demoing. The cosine similarity translation function was flawed (1 - L2_distance rather than 1 - L2_distance² / 2), resulting in a severe underestimation of every similarity score. Searches that were expected to yield a score of 0.80 to 0.93 returned scores ranging from 0.36 to 0.45. Why did the HVAC demo succeed? Because all three demo searches were chosen by hand, at threshold levels that seemed appropriate. However, the evaluation suite revealed that 30% of the ground-truth searches were being mis-classified due to the incorrect similarity function.

A demo is not a proof-of-concept; an evaluation suite is. The motivation to demonstrate a working system on three queries carefully selected beforehand is high; however, the discipline to perform 85 evaluations prior to release is higher.

9.2 Confidence Thresholds Are the Product

At first, I framed this as a purely engineering optimization challenge: Find the (high_threshold, partial_threshold) pair that produces the highest eval score. But after conducting the adversarial test suite, my perspective changed. Increasing the partial threshold to make more aggressive escalation would yield a better adversarial score but would worsen the ground truth score.

Thinking about threshold setting as product governance, not engineering, is the key product insight I'd emphasize at the beginning of any production discussion.

9.3 What I Would Revisit

Three design decisions I would revisit in Phase 3:

1. Retrieval through dense vectors only. The reasons for adversarial cases are due to our reliance on dense vectors only for retrieval. If hybrid retrieval had been implemented, the adversarial success rates would be from 45% to 70%+ more readily achieved. Starting with the dense-only retrieval system was important.
2. Fixed token chunking. Documents are all chunked at fixed token positions. This causes chunks of tabular data to be fragmented into separate chunks, making retrieval of such information challenging for specification queries. The semantic or structural chunking technique will solve this problem.
3. Regression test separation between drone and HVAC assistants. We currently have a regression test suite that checks for the HVAC assistant and another suite that tests the drone assistant. It would be better to have a unified regression test suite testing both entities post any change to catch regression cases.

10. Phase 3 Vision & Roadmap

10.1 Multi-Graph Memory (MAGMA-Inspired)

Phase 3 aims at a reasoning level on a longitudinal scale, which the existing architecture is unable to accommodate. The existing session memory stores information about the entities within the session; it does not span multiple sessions and also fails to perform longitudinal reasoning. The technician visiting Zone C after three months would be unable to query the system whether the pattern of corrosion has altered since August, and receive a valid response.

The proposed architecture uses four different types of memory graphs:

1. Temporal: Timestamped and sequenced anomaly readings facilitate trend analysis.
2. Causal: Relationships between discoveries and fixes (for example, Zone B corrosion > Fix > Inspect).
3. Entity: Persistent entity identifiers for machines and zones.
4. Semantic: Topic-based clustering of the entire corpus.

10.2 Phase Roadmap

Phase	Status	Key Deliverable
Phase 1 - HVAC Foundation	Complete	RAG pipeline, graceful degradation, 85-case eval suite
Phase 2 - Drone Extension	In Progress	Classifier agent, spatial filtering, session memory, dual-domain Streamlit app
Phase 3 - Production + Memory	Planned	Deployed URL, multi-graph memory, longitudinal anomaly tracking

Moreover, Phase 3 involves querying the pipeline for its first true multi-domain inquiry where the tech is posing a question relating to the inspection performed by the drone against an OSHA rule and doing so in one single query. This necessitates the need for query routing.

11. Appendix

A. Evaluation Results Sample

Set	Query (excerpt)	Expected	Actual	Pass	Score
ground_truth	Lockout tagout energy control procedure	HIGH/PARTIAL	HIGH	True	0.933
ground_truth	Installation requirements for Trane XR15	HIGH/PARTIAL	LOW	False	0.482
adversarial	Repair procedures for Daikin VRV DX300	LOW	LOW	True	0.310
adversarial	Refrigerant charge for Rheem RA20	LOW	PARTIAL	False	0.564
adversarial	OSHA requirements for confined space entry	LOW	PARTIAL	False	0.762
contradictions	Electrical safety during HVAC maintenance	PARTIAL	PARTIAL	True	0.666

Set	Query (excerpt)	Expected	Actual	Pass	Score
contradictions	Carrier 48LC filter intervals 2017 vs 2023	PARTIAL	LOW	False	0.484

B. Technology Stack

Layer	Technology
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Vector database	Chroma (local persistence)
LLM providers	Google Gemini 2.5 Flash (primary), Anthropic Claude 3.5 Sonnet, OpenAI GPT-4o
Framework	Python 3.11, LangChain Chroma + HuggingFace adapters
UI	Streamlit (6-page guided walkthrough)
Data generation	Claude API (synthetic HVAC jobs + drone inspection records)
Eval	Custom Python runner, eval_results.csv output

C. Key Design Constraints

1. Do not aggregate data sets. Conflict detection via sources requires understanding where each result originated.
2. All responses must provide a source. No sources = no answer. Guaranteed by the system prompt.
3. LOW-level paths do not invoke the LLM. Escalation is handled programmatically.
4. Confidence levels are set per product. Alterations require approval post-eval-suite review.